Rope Path Visualization in FreeCAD Assembly Workbench

A Technical Summary

Problem Statement

FreeCAD's Assembly Workbench (v1.0+) has no built-in support for rope or cable visualization. This document describes a Python macro approach that computes geometrically correct rope paths — including tangent lines and arcs — across multiple pulleys in a 3D assembly, with automatic updates when the assembly moves.

Key Concepts

1. Local Coordinate Systems (LCS) as Rope Reference Points

Each rope-relevant part carries one or more `Part::LocalCoordinateSystem` objects **inside its PartDesign Body**. This is critical: the LCS must be a child of the Body so it travels with the part when inserted as a link into an assembly.

Naming convention:

LCS name	Purpose
`LCS_rope_groove`	Pulley groove centre
`LCS_winch`	Rope exit point on drum
`LCS_anchor`	Fixed rope end (knot)

Axis convention: X-axis = rotation axis of the pulley (consistent with CadQuery and FreeCAD Part Design defaults for revolve features).

Properties on the LCS (added via the Properties panel, group `Eitech`):

Property	Type	Meaning
`radius`	Length	Groove bottom radius (geometry of the part)

The rope diameter is **not** stored on the LCS — it belongs to the rope object in the assembly (property `rope_diameter` on the wire feature).

Effective radius used in calculations:

```
r_eff = lcs.radius.getValueAs("mm") + rope_diameter / 2
```

---

2. Accessing LCS Data from a Linked Part

Parts are inserted into assemblies as `App::Link` objects. To read the LCS:

python

```python
def get_lcs_data(link_obj, lcs_label, rope_diameter_mm):
    linked = link_obj.LinkedObject          # the original part

    # Search for LCS inside the part (direct child or inside a Body)
    lcs = None
    for child in linked.OutList:
        if child.Label == lcs_label:
            lcs = child; break
        if hasattr(child, "OutList"):
            for subchild in child.OutList:
                if subchild.Label == lcs_label:
                    lcs = subchild; break
        if lcs: break

    # Global transformation: assembly placement × LCS local placement
    global_pl = get_global_placement(link_obj).multiply(lcs.Placement)
    pos   = global_pl.Base                                    # position
    achse = global_pl.Rotation.multVec(App.Vector(1, 0, 0))  # X-axis = rotation axi

    r_mm  = lcs.radius.getValueAs("mm")
    r_eff = r_mm + rope_diameter_mm / 2 if r_mm > 0 else 0.0
    return pos, achse, r_eff
```

Important: Always use `.getValueAs("mm")` to extract a plain float from a FreeCAD `Quantity`. Direct arithmetic with Quantities raises `ArithmeticError` due to unit mismatches.

---

3. Global Placement in Nested Assemblies

`App::Link` objects only carry their placement relative to their immediate parent. For nested assemblies, the global placement must be computed by traversing the `InList` chain:

python

```python
def get_global_placement(link_obj):
    placement = link_obj.Placement
    current   = link_obj

    for _ in range(10):                      # max nesting depth
        in_list = current.InList
        if not in_list:
            break
        parent = in_list[0]
        if not hasattr(parent, "Placement"):
            break
        if parent.TypeId == "Assembly::AssemblyObject":
            current = parent                 # internal container, Placement=(0,0,0)
            continue
        placement = parent.Placement.multiply(placement)
        current = parent

    return App.Placement(placement)
```

Key insight: `Assembly::AssemblyObject` is the internal container of a sub-assembly file — its placement is always (0,0,0) and must be skipped. The `Assembly::AssemblyLink` one level up carries the actual global placement.

Note: `getGlobalPlacement()` is not available on `App::Link` objects in FreeCAD 1.1. The `InList` traversal is the reliable alternative.

---

4. Tangent Line Calculation

The rope segment between two pulleys must satisfy:

```
R1 ⊥ A1,   S ⊥ R1,   S ⊥ R2,   R2 ⊥ A2
```

where `A1`, `A2` are the pulley axes, `R1`, `R2` are the radius vectors to the tangent points, and `S = T2 - T1` is the rope segment vector.

This leads to:

```
R1 = r1 * norm(A1 × S) * w1
R2 = r2 * norm(A2 × S) * w2
```

where `w1`, `w2` ∈ {+1, -1} are the **winding signs** of the respective pulleys.

Sign convention: `w = +1` if the pulling rope produces a positive torque about the pulley axis (right-hand rule).

Since `S` appears on both sides, the solution is iterative:

```python
def berechne_tangente(M1, A1, r1, w1, M2, A2, r2, w2,
                      max_iter=50, toleranz=1e-6):
    S = M2 - M1                              # initial guess
    for _ in range(max_iter):
        R1 = r1 * w1 * norm(cross(A1, S)) if r1 > 0 else (0,0,0)
        R2 = r2 * w2 * norm(cross(A2, S)) if r2 > 0 else (0,0,0)
        T1 = M1 + R1
        T2 = M2 + R2
        S_new = T2 - T1
        if |S_new - S| < toleranz: return T1, T2
        S = S_new
```

Convergence is typically achieved in 1–6 iterations for realistic pulley geometries. The algorithm works for arbitrary axis orientations in 3D — parallel, skew, or perpendicular axes all converge correctly.

The winding sign belongs to the pulley, not to the rope segment. Each segment uses `w1` from the departure pulley and `w2` from the arrival pulley:

```python
SEILPFAD = [
    ("Winch",    "LCS_winch",       -1),   # winding sign of this pulley
    ("Pulley_A", "LCS_rope_groove", +1),
    ("Pulley_B", "LCS_rope_groove", -1),
    ("Anchor",   "LCS_anchor",       0),   # endpoints: no arc
]
```

---

5. Arc Calculation on Pulleys

At each intermediate pulley, the rope wraps around an arc. The arc must be **geometrically continuous** with the incoming and outgoing rope segments.

Method: build a local 2D coordinate system in the pulley plane:

```python
R_ein = norm(T_ein - M)          # unit radius vector to entry point
e1    = R_ein                    # 0° reference
e2    = norm(cross(A, e1))       # 90° in the plane, perpendicular to A and e1

theta_aus = atan2(dot(R_aus, e2), dot(R_aus, e1))   # exit angle
```

The winding sign `w` determines the traversal direction:

- `w = +1`: counter-clockwise (theta increases)
- `w = -1`: clockwise (theta decreases)

```python
if w > 0 and theta_aus <= 0: theta_aus += 2*pi
if w < 0 and theta_aus >= 0: theta_aus -= 2*pi

theta_mid = theta_aus / 2        # midpoint of arc
T_mid = M + r * (cos(theta_mid)*e1 + sin(theta_mid)*e2)
```

Continuity check: verify that the arc tangent at `T_ein` points in the same direction as the incoming rope segment `S_ein`. The arc tangent at the entry point is `e2 * w` (perpendicular to `R_ein` in the winding direction):

```python
arc_tangent = e2 * w
if dot(norm(S_ein), arc_tangent) < 0:
    w = -w                       # reverse arc direction
    # recompute theta_aus and theta_mid with new w
```

This automatic correction handles cases where the initial winding sign produces a discontinuous path — the arc is simply replaced by its complement. This is robust for all wrap angles including ~0°, ~180°, and beyond — wrap angles greater than 180° occur in practice and are handled correctly by the explicit angle calculation (the arc midpoint at `theta_aus / 2` always lies on the correct arc regardless of wrap angle).

The arc is constructed using `Part.Arc(T_ein, T_mid, T_aus)`.

---

6. Wire Assembly

The complete rope path is:

```
straight(0→1) → arc(pulley 1) → straight(1→2) → arc(pulley 2) → ... → straight(n-1→n)
```

Each segment endpoint coincides exactly with the next segment's start point, so `Part.Wire(edges)` succeeds without gaps.

---

7. Update on Request via DocumentObserver

```python
class SeilObserver:
    def slotRecomputedDocument(self, doc):
        draw_seil(doc, do_recompute=False)   # False prevents recursive recompute

App.addDocumentObserver(SeilObserver())
```

The observer fires after each document recompute, which is triggered by pressing the Refresh button in the toolbar. The rope updates on demand — not continuously during mouse drag, but immediately after releasing and pressing Refresh. The `do_recompute=False` flag is essential — calling `doc.recompute()` inside a recompute callback causes a recursive loop.

To make `stop_observer()` accessible from the console after running as a macro, store the observer in the global `App` namespace:

```python
App._eitech_observer = observer
```

Demo Script

The following script creates a minimal test assembly in a new FreeCAD document:
two pulleys and an anchor point, computes the rope path, and visualizes it.
Run it in the FreeCAD Python console or as a macro.